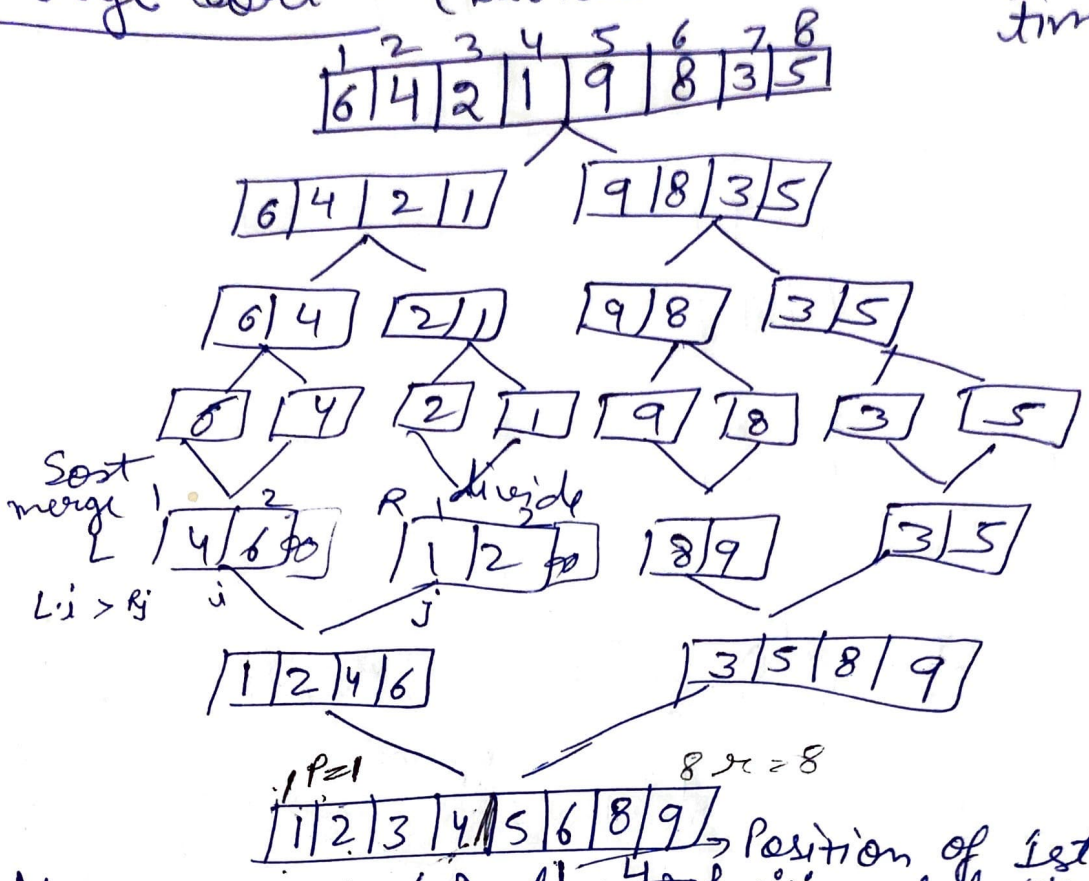
Merge Sort – (Divide & conquer)

time complexity
n log n



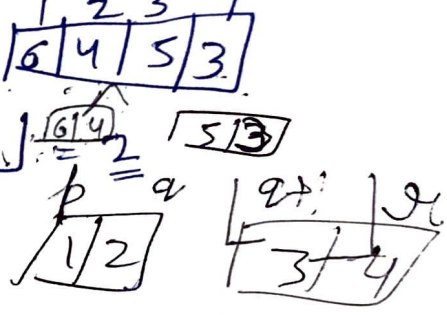
Position of 1st element

Position of last element

Merge sort (A, p, r)

1- if $p < r$

2- $q = \lfloor (p+r)/2 \rfloor$ $= \lfloor (1+4)/2 \rfloor = 2$



3- Merge-sort (A, p, q)

4- Merge-sort (A, q+1, r)

5- Merge (A, p, q, r)

↳ 1- $n_1 \leftarrow q - p + 1$; (2 − 1 + 1 = 2)

2- $n_2 \leftarrow r - q$ (4 − 2 = 2)

3- Create array L(1 ... n_1+1) and R[1 ... n_2+1]

4- for $i \leftarrow 1$ to n_1

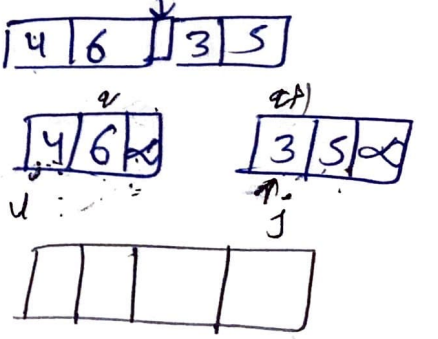
5- do $L[i] \leftarrow A[p+i-1]$

6- for $j \leftarrow 1$ to n_2

7- do $R[j] \leftarrow A[q+j]$

8- $L[n_1+1] \leftarrow \infty$

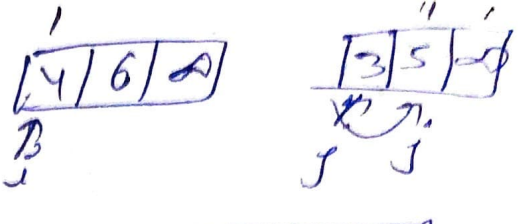
9- $R[n_2+1] \leftarrow \infty$



$[1+1-1]$

$L[i] \leftarrow A[1]$

10- $i \leftarrow 1$
11- $j \leftarrow 1$
12- for $k \leftarrow p$ to r
13- do if $L[i] \leq R[j]$
14- then $A[k] \leftarrow L[j]$
15- $i++$
16- else $A[k] \leftarrow R[j]$
17- $j++$



Quick sort → (DAE)

Pievert element

35, 5, 4, 3, 2, 1

35 50, 15, 25, 80, 20, 90, 45 +∞

p must check ≤ elmnt; if not cross each other swap the elm; q must check element ≤ elmnt

35, 20, 15, 25, 80, 50, 90, 45 +∞

p → ; q stop; p stop ←; q stop

if P & Q crosses each other then replace the value of pievert element with Q.

lesser ← → greater.

25, 20, 15, 35, 80, 50, 90, 45 +∞

q; q; p

25, 20, 15 +∞

p → ; q; p

15, 20, 25 35

80 50 90, 45 +∞

p; q

80 50 45 90 +∞

p; q

Algorithm of Quick Sort — (A, p, r)

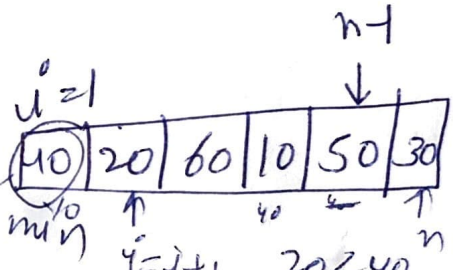
1- if p < r then
2- q ← Partition (A, p, r)
3- Quick-Sort (A, p, q-1)
4- Quick-Sort (A, q+1, r)

Partition (A, p, r)

1- x ← A[r]
2- i ← p-1
3- for j ← p to r-1
4- do if A[j] ≤ x
5- then i ← i+1
6- exchange A[i] ↔ A[j]
7- exchange A[i+1] ↔ A[r]
8- return i+1

Selection Sort —

1- for i ← 1 to n-1 do
2- min j ← i;
3- min x ← A[i]
4- for j ← i+1 to n do
5- if A[j] < min x then
6- min j ← j
7- min x ← A[j]
8- A[min j] ← A[i]
9- A[i] ← min x



min = 20

min = 10

1st pass -

i=2 n-1

min = 60

Hashing – (Storing & Retrieving data in O(1) time)

* Search Key (24, 52, 91, 67, 48, 83)
* Hash table
* Hash function (k mod 10, k mod n, Mid Square, folding method)

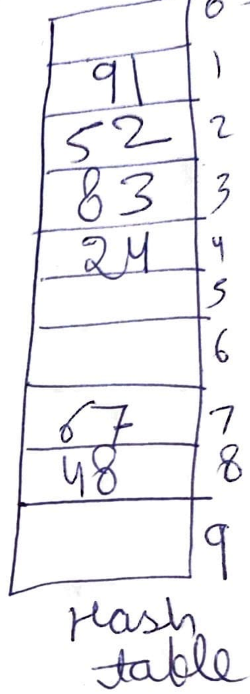
k mod 10
24 mod 10
Hash value = 4
n = no. of keys

123
Mid square
$2^2 = 4$

folding method 123456
0 --- 999

123
+ 456
579 loc

Search 67 mod 10



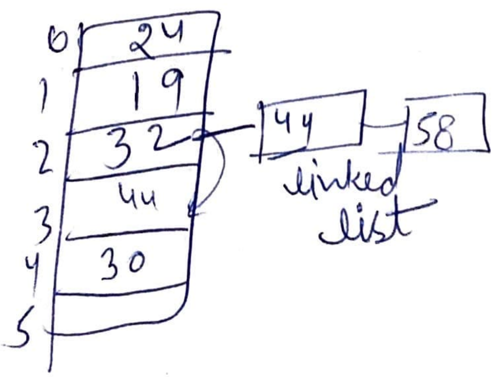
Collision Resolution in Hashing – techniques

Chaining (Open Hashing) – linked list

Open Addressing (Closed Hashing)
- Linear Probing
- Quadratic Probing
- Double Hashing

Next available space
min daling

k mod 6
(24, 19, 32, 44)



30

$h + i^2 \bmod n$

$0 + 1^2 \bmod 6$

$0 + 2^2 \bmod 6$ = 4